QBioS Laboratory: The Geometry of Gaits

Joshua S. Weitz[1, 2, *]

[1] School of Biology, Georgia Institute of Technology, Atlanta, GA, USA
[2] School of Physics, Georgia Institute of Technology, Atlanta, GA, USA

(Dated: October 14, 2020)

I. INTRODUCTION

This laboratory culminates the organismal and physiology module. It comes full circle in a sense, from the first laboratory that covered *E. coli* dynamics from the perspective of chemotaxis. There, the objective was to represent the kinase cascade that transformed external signals into changes in the run and tumble motion of the microbe. Hence, the laboratory focused largely on how microbial cells modulate their behavior in light of environmental gradients. Yet, the behavior was taken for granted. Somehow, *E. coli* moves. Likewise, many other microbes, cells, and organisms at scales from microns to meters in length manage to explore their environments with gaits, behaviors, and movements that often share surprisingly common principles. This chapter will take inspiration from the humble *E. coli* and focus in on one particular limit: when damping (aka resistive aka viscous) forces are so large that they overwhelm the inertial forces. In doing so, it is possible to understand how organisms move, by generating internal forces that change their own body space configuration (and center of mass), thereby interacting with the environment.

In doing so, this laboratory will center on Purcell's 3-link swimmer (see image below). The three-link swimmer has two internal degrees of freedom, α_1 and α_2 denoting the angles of the two side links relative to that of the center link. The angles of these links are controlled by the swimmer, and we will assume that the swimmer always has sufficient force to (precisely) control changes in its internal configuration, i.e., $(\alpha_1(t), \alpha_2(t))$. If these internal dynamics repeat, i.e., $\alpha_1(t) = \alpha_1(t+T)$ and $\alpha_2(t) = \alpha_2(t+T)$, we can call such periodic orbits a 'gait' in configuration space. But orbits in configuration spaces don't tell the complete story. The objective of this laboratory is connect shape deformations to motion in the world frame, what is denoted as (x, y, θ) for the 3-link swimmer (see image below). By focusing on this humble 3-link swimmer, this laboratory is a gateway to a larger question: understanding how an organism that moves a flagella, undulate its body, or flaps its wings manages to do more than just flail helplessly but instead to

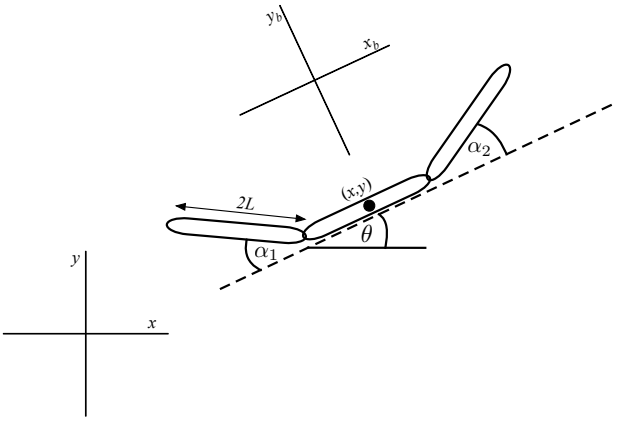


FIG. 1: Three-link swimmer with limbs that can move given a contact point to the body via two angles, α_1 and α_2. The angle of the swimmer in the world frame is θ. The center-point position of the swimmer in the world frame is (x, y). In the body frame, any point on the swimmer can be designated by the coordinated (x_b, y_b), such that in the world frame all points can be written as $(x + x_b, y + y_b)$.

*Electronic address: jsweitz@gatech.edu; URL: http://ecotheory.biology.gatech.edu

Typeset by REVTEX

swim, dig, or soar into flight.

This field is mathematically rich and challenging. The conventional approach is to consider the zero-inertia limit in which an organism cannot move unless it continues to generate internal body forces, i.e., the no-coasting limit. In that sense, one must solve for the steady state velocity consistent with the absence of inertial forces given a particular gait. Doing so opens the door to notions of geometric phase and other profound insights that transcend the particular system (i.e., the ideas hold for both microbes swimming in water as they do for metazoans 'swimming' in sand). However, this path requires an understanding of Navier Stokes equations as well as a familiarity with notions of gauge theory. Instead, we will take a dynamical systems perspective and develop a computational model that incorporates basic Newtonian physics – think $F = ma$ – with internal gait programs – think $\alpha_i(t)$ – as a means connect internal force generation with movement in the world frame. As described in the textbook, this lab bridges the paradigms of Newton and Borelli, and hopefully brings simple physical ideas of dynamical motion to life.

II. THE INTERNAL ORIGINS OF MOVEMENT

Consider a 3-link swimmer whose position and orientation in the world frame are described by (x, y, θ) where x and y denotes the position of the centerpoint of the body and θ denotes the angle of the center link relative to the x-axis. Hence, there are many configurations of the swimmer irrespective of where it is located in space, or how it is rotated. The internal configuration space represent the menagerie of shapes that the 3-link swimmer can adopt. For example, when both $\alpha_1 = -\pi/2$ and $\alpha_2 = -\pi/2$, the configuration looks likes a paperclip. While, when $\alpha_1 = -\pi/2.5$ and $\alpha_2 = \pi/2$ the configuration looks like someone in a leather reclining chair. Indeed, viewing these configurations is central to the concept of gaits – which represent orbits in configuration space and therefore shifts in internal body shape, i.e., the positioning of links relative to one another.

Let's see how. Rather than visualize nicely arced ovals for links, let's instead aim for simplicity. Links will be represented as lines in space. In order to do so, it will be essential to calculate the positions of each of the 4 nodes that connect 3 links in the body frame: (x_b, y_b). This function is termed `body_coords`, and returns a set of position pairs given links each of length $2L$ and two angles:

```
import numpy as np
import matplotlib.pyplot as plt

def body_coords(alpha,L):
    """
    Converts two body angles into the four positions of the 3 link swimmer
     - in their own body frame
    """
    dx =-np.cos(alpha[0])
    dy = np.sin(alpha[0])
    pos = np.zeros((4,2))
    pos[0,:] = 2*L*np.array([dx,dy])+np.array([-L, 0])
    pos[1,:] = np.array([-L,0])
    pos[2,:] = np.array([L,0])

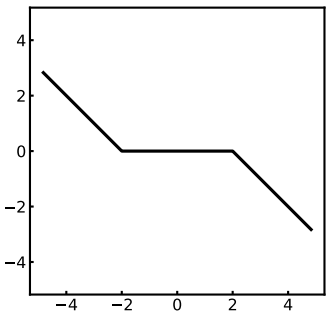
    dx = np.cos(alpha[1])
    dy = np.sin(alpha[1])
    pos[3,:] = np.array([L,0])+2*L*np.array([dx, dy])

    return(pos)
```

Once you have this code working, try out different configurations and then plot the 3-link swimmer, e.g.,

```
fig = plt.figure(figsize=(5,5))
ax = fig.gca()
pos = body_coords([np.pi/4, -np.pi/4],2)
plt.plot(pos[:,0],pos[:,1],'k',linewidth=3)
ax.axis('equal') #To fix the aspect ratio
```

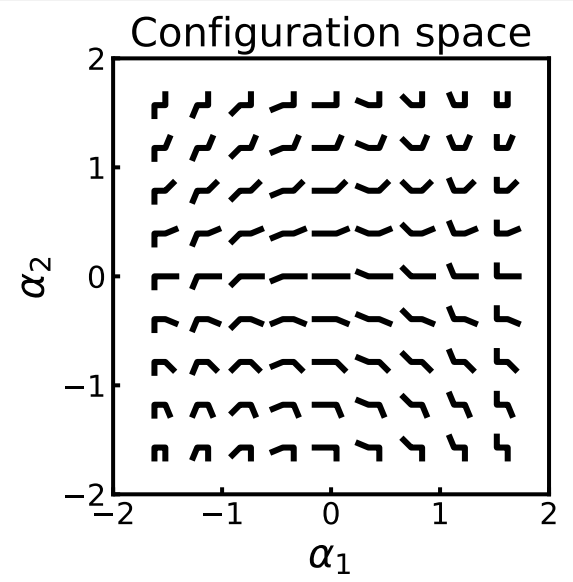
Try out the code to make sure it does the right thing for different angles and for different link lengths. If it's working it should look like:



Now you're ready for the first challenge problem: to write a code that can generate the configuration space in which the 3-link swimmer will operate.

Challenge Problem: Visualizing Configuration Space

Extend the code snippet to visualize a single body configuration as a configuration space from $-\pi/2 \leq \alpha_1 \leq \pi/2$ and $-\pi/2 \leq \alpha_2 \leq \pi/2$. If your code works, you should see an image that looks like the following:



Solutions to Challenge Problem: Visualizing Configuration Space
There are many ways to solve this problem. This solution set uses a vector of values, generates a 'mesh' of (α_1, α_2), and then finds the body coordinates for each set of angles. It is up to you to decide how finely to divide up the configuration space, here the space includes 9 values of α_1 and α_2, for 81 potentially different configurations:

```
fig = plt.figure()
ax =  fig.gca()
L = 0.05 # Artificially create 'small' swimmers for visualization
dax = np.pi/8
day = np.pi/8
alpha1_vec = np.arange(-np.pi/2,np.pi/2+dax,dax)
alpha2_vec = np.arange(-np.pi/2,np.pi/2+day,day)
alpha1, alpha2 = np.meshgrid(alpha1_vec, alpha2_vec)

for i in range(len(alpha1_vec)):
    for j in range(len(alpha2_vec)):
        pos = body_coords([alpha1[j,i],alpha2[j,i]],L) #Configuration
        vis_pos = pos + np.array([alpha1[j,i],alpha2[j,i]]) # Shift placement to alpha-1/alpha-2
        plt.plot(vis_pos[:,0],vis_pos[:,1],'k',linewidth=3)
plt.xlabel(r'$\alpha_1$',fontsize=20)
plt.ylabel(r'$\alpha_2$',fontsize=20)
plt.title('Configuration space',fontsize=20)

plt.xlim(-2,2)
plt.ylim(-2,2)
ax.set_aspect('equal','box')
plt.setp(ax.spines.values(),linewidth=2)
ax.tick_params(labelsize=15,direction='in',width=2)
```

III. ORBITS IN CONFIGURATION SPACE

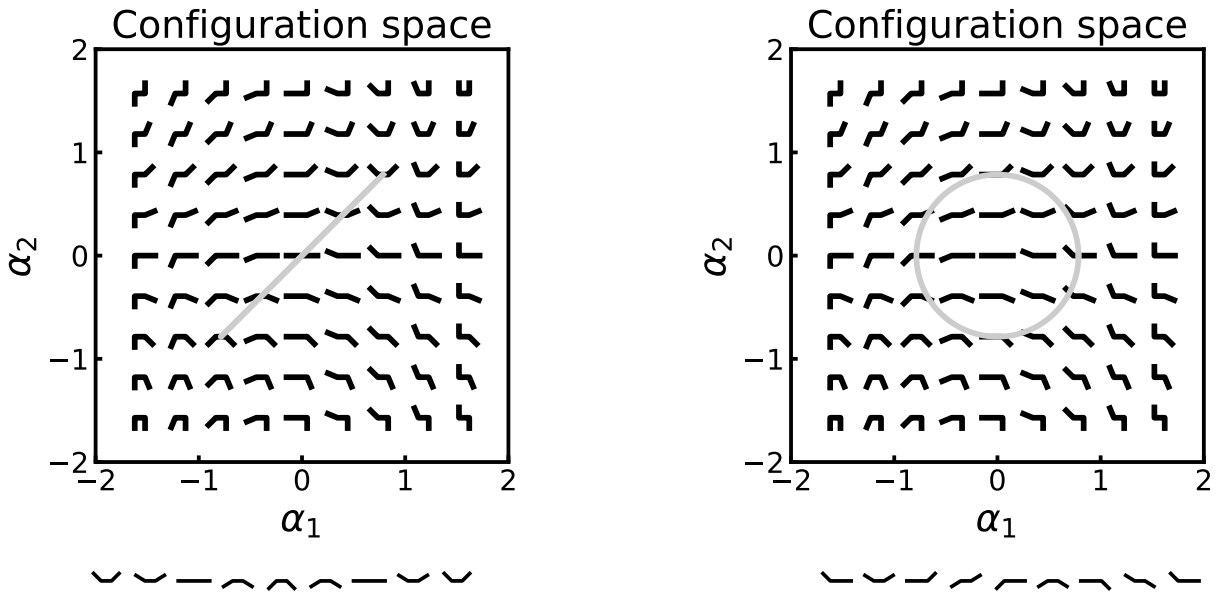
Before moving to the full Newtonian framework, it is essential to master the concept of orbits in configuration space, aka gaits. A gait is a repeated set of movements. For example, if the 3-link swimmer were to move its links up and down, up and down, and so on, this can be thought of as a symmetric gait that traverses a line twice in the configuration space, albeit in opposite directions. Familiarizing yourself with the relationship between paths in configuration space and a representative sequence of configurations that make-up a gait is essential to thinking about how organisms move in practice – whether for *E. coli*, spermatazoa, or sand lizards. To get you started, consider the following symmetric gait function:

```
def sym_gait(t,T):
    """
    Returns positions and velocities in angle space for customized gait in a
    single python list
    """
    alpha1 = np.pi/4*np.cos(2*np.pi*t/T)
    alpha2 = np.pi/4*np.cos(2*np.pi*t/T)
    valpha1 = -2*np.pi/T*np.pi/4*np.sin(2*np.pi*t/T)
    valpha2 = -2*np.pi/T*np.pi/4*np.sin(2*np.pi*t/T)
    return [alpha1,alpha2,valpha1,valpha2]
```

This gait can be superimposed over the configuration space by adding the following code:

```
#Overlay the gait
gait_dyn = sym_gait(np.arange(0,1.01,0.01),1)
alpha1_gait = gait_dyn[0]
alpha2_gait = gait_dyn[1]
plt.plot(alpha1_gait,alpha2_gait,'r',linewidth=3)
```

This code yields a 'gait'. The gait is denoted as the gray line in the configuration space. In the left-panel that gait winds back over itself, that is the gait starts with both α_1 and α_2 near 1 and then near -1 and then near 1 and so on. This is depicted in the left-panel of the image set below. In the next challenge problem, it is your job to figure out a gait that can reproduce the sequence of configurations in the right panel.



Challenge Problem: Designing Gaits

Develop an anti-symmetric gait that reproduces the motions above and (optionally) a square-wave shape of an anti-symmetric gait. If you have the time, write a code snippet that reproduces the configuration snapshots below the top panels in the prior figure.

Solutions to Challenge Problem: Designing Gaits (Part 1 of 2)
This solution has multiple components – first the gaits, which can be coded up as follows:

```
def asym_gait(t,T):
    """
    Returns positions and velocities in angle space for customized gait in a
    single python list
    """
    alpha1 = np.pi/4*np.cos(2*np.pi*t/T)
    alpha2 = np.pi/4*np.sin(2*np.pi*t/T)
    valpha1 = -2*np.pi/T*np.pi/4*np.sin(2*np.pi*t/T)
    valpha2 = 2*np.pi/T*np.pi/4*np.cos(2*np.pi*t/T)
    return [alpha1,alpha2,valpha1,valpha2]
```

Solutions to Challenge Problem: Designing Gaits (Part 2 of 2)
A gait sequence be plotting using the following code snippet which turns off the axis to highlight the internal configuration changes.

```
fig = plt.figure()
ax =  fig.gca()

for t in np.arange(0,1.125,0.125):
    gait_dyn = sym_gait(t,1)
    alpha1 = gait_dyn[0]
    alpha2 = gait_dyn[1]
    pos = body_coords([alpha1,alpha2],0.015)
    vis_pos = pos + np.array([t, 0])
    plt.plot(vis_pos[:,0],vis_pos[:,1],'k',linewidth=3)
plt.xlim(-0.1,1.1)
plt.ylim(-0.2,0.2)
plt.axis('off')
```

IV. FROM BORELLI TO NEWTON AND BACK AGAIN

A. Newtonian framework

How do we move from gait programs $(\alpha_1(t), \alpha_2(t))$ to dynamics in the world frame? The tack we will take here is to focus on classical mechanics, i.e, $F = ma$ where F, m, and a denote force, mass, and acceleration. Conventionally physics classes emphasize how simple physical interactions can be described in terms of 2nd order (usually linear) differential equations. However, such equations can be written in terms of a coupled set of first-order differential equations where the state space includes both position and velocity. That logic underlies the development of the Fitzhugh-Nagumo model (introduced in the prior chapter). Here, the same logic applies, albeit to the position (x, y), orientation, θ, and corresponding velocities, i.e., translational velocities (v_x, v_y) and angular velocity, ω_θ. Recognizing that positions changes due to velocities and velocities change due to forces motivate the following set of equations for the dynamics of the 3-link swimmer in the world frame:

$$\dot{x} = v_x \tag{1}$$
$$\dot{y} = v_y \tag{2}$$
$$\dot{\theta} = \omega_\theta \tag{3}$$
$$\dot{v}_x = \frac{F_x}{m} \tag{4}$$
$$\dot{v}_y = \frac{F_y}{m} \tag{5}$$
$$\dot{\omega}_\theta = \frac{\tau}{I}. \tag{6}$$

Here, F_x and F_y denote forces, M denotes the total mass, τ is the torque, and I is the moment of inertia.

These equations seem straightforward, but they also seem to hide the key insight: that changes in the internal body configuration change the dynamics in the world frame. In essence, where is $\alpha_1(t)$ and $\alpha_2(t)$? The answer is embedded in a simple principle, for every action there is an equal and opposite reaction. Consider a particle moving with velocity v facing a drag k, such that the total force is $F = -kv$. In that event, one could write a simplified set of equations:

$$\dot{x} = v \tag{7}$$
$$\dot{v} = -kv/m. \tag{8}$$

These equations are the very core of how to understand how the 3-link interacts with its viscous environment (and, similarly how the sandfish interacts with its dissipative, granular environment). In general, an organism moving with an initial velocity v_0 will coast for a while until to comes to a complete stop at position x_f. For the 3-link swimmer, this coasting distance (and indeed the effective coasting time, τ) set both length and time scales to compare to the

length of the organism and the duration of the gait, T. In the limit that $x_f \ll L$ and $\tau \ll T$ then we should expect that however the organism travels, it must be primarily due to internal body forces rather than due to inertia (and coasting). This insight underlies the next challenge problem.

Challenge Problem: Coasting Time and Distance

Consider a non-motile organism of mass m with an initial velocity of v_0 in an environment with drag k. How far will the organism coast before it reaches a complete stop? What is the effective time over which the organism remains in motion? In answering this problem, use a simulation and (if possible) mathematical analysis to verify your finding.

Solutions to Challenge Problem: Coasting Time and Distance

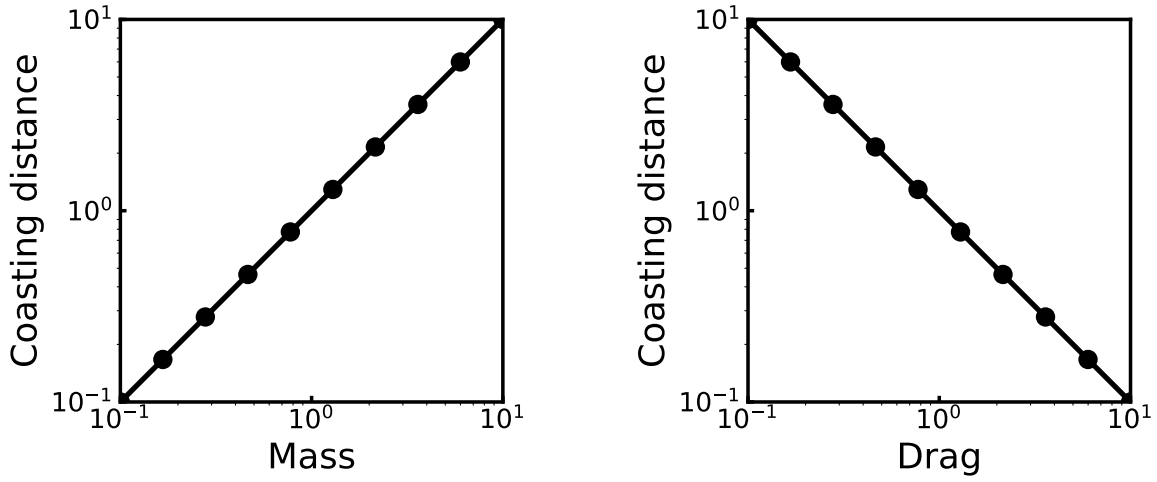
This coupled set of linear differential equations can be solved, first by recognizing that $v(t) = v_0 e^{-kt/m}$. As a result $\dot{x} = v_0 e^{-kt/m}$, which implies that

$$\mathrm{dx} = \mathrm{v_0 e^{-kt/m} dt} \tag{9}$$

Integrating both sides yields

$$x(t) = -\frac{v_0 m}{k}\left(e^{-kt/m}\right)\Big|_0^\infty \tag{10}$$

such that $x(t) = \frac{v_0 m}{k}$. Hence, larger organisms will coast longer, likewise less viscous environments (with less drag) will also lead to longer coasting distances. Note that the effective time-scale of coasting is $\tau = m/k$. The following code simulates the model and verifies the relationship as shown in the figure below (either for variation in m or in k:



The key take-away here is that the ratio of m/k sets a time-scale (and therefore a coasting distance) over which inertia can act. When $(m/k) \ll T$, the gait period, then it must be internal forces, rather than inertial forces, that control propulsion.

Solutions to Challenge Problem: Simulation code for Coasting Time and Distance

Dynamic code

```
def model_coast(y,t,pars):
    dydt = np.zeros(2)
    dydt[0] = y[1]
    dydt[1] = -pars['k']*y[1]/pars['m']
    return dydt
```

Simulation and visualization code

```
# Simulation
mrange = np.logspace(-1,1,num=10)
v0=1
pars={}
yf = np.zeros(np.shape(mrange))
for i in range(len(mrange)):
    pars['m']=mrange[i]
    pars['k']=1
    t = np.linspace(0,100)
    y0 = np.array([0,v0])
    y = integrate.odeint(model_coast,y0,t,args=(pars,))
    yf[i] = y[-1,0]

#plot simulation against theory
fig = plt.figure()
ax = fig.gca()
m = np.logspace(-1,1,num=1000)
plt.loglog(m,m*v0/pars['k'],'k',linewidth=3)
plt.loglog(mrange,yf,'k',
           linestyle='',
           marker = 'o',
           markersize = 10)
plt.xlabel('Mass',fontsize=20)
plt.ylabel('Coasting distance',fontsize=20)
```

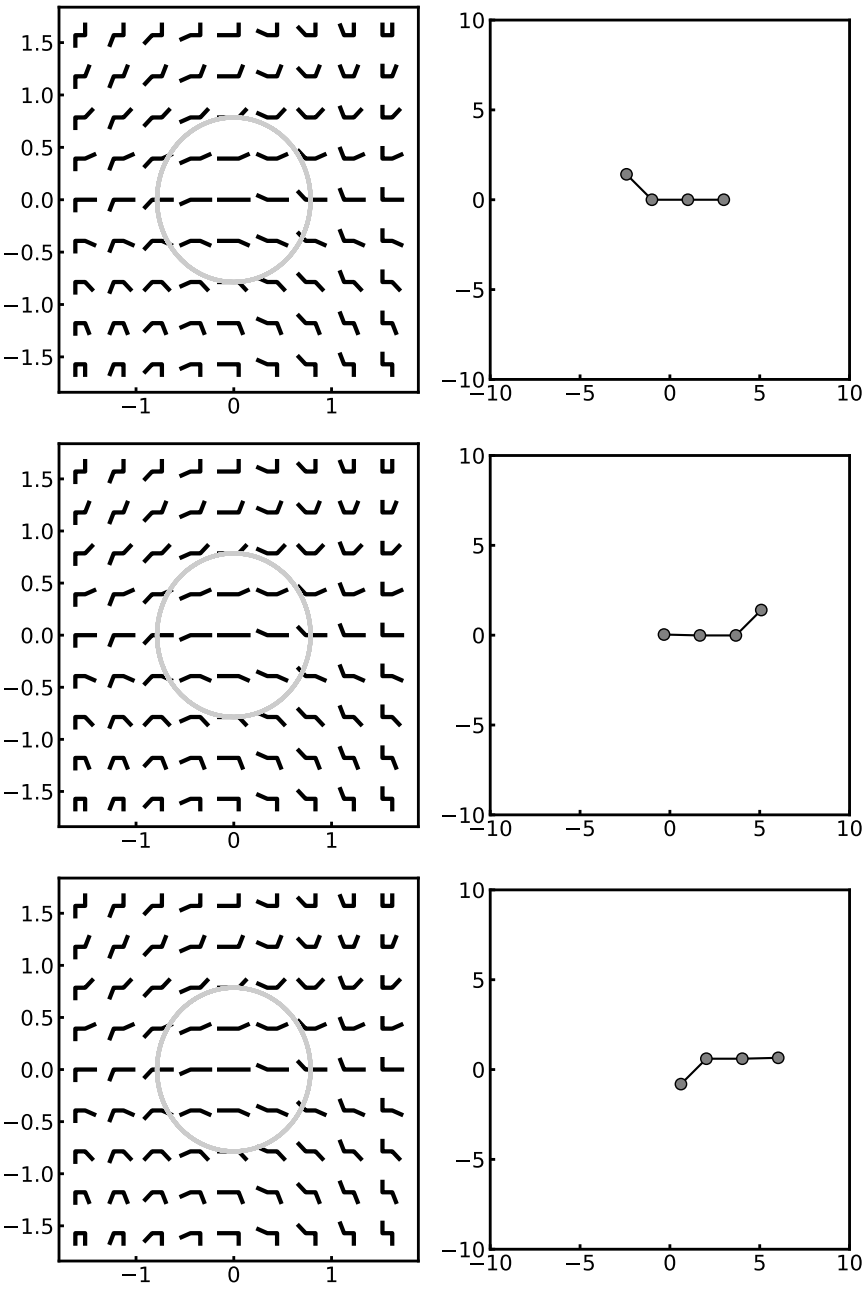
B. Borelli: Connecting External Forces and Internal Body Movements

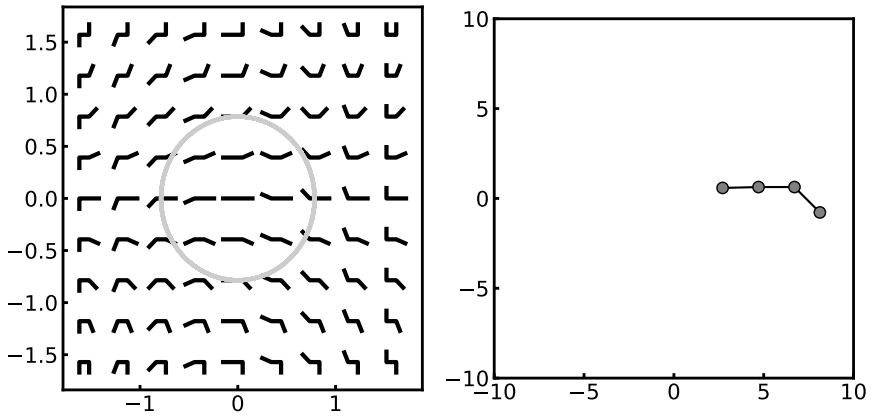
In the limit that inertial forces are small and that coasting is minimal, then the internal movement of the body is the key to understanding how the swimmer self-propels. Indeed, it is 'drag' that both slows the organism down and allows it to move. Because, by moving a link in one direction the organism must experience an equal force in the opposite direction. We have already assumed that $F_{drag} = -kv$. However, here we will make the critical assumption that $F_{propulsion} = -kv_{link}$. In other words, if the link moves backwards, then the force acting on the body is forward. Likewise, if the link moves forwards, then the force acting on the body is backwards. Hence, we must break-down the forces into two components:

$$F_x = F_x(drag) + F_x(propulsion) \tag{11}$$
$$F_y = F_y(drag) + F_y(propulsion) \tag{12}$$

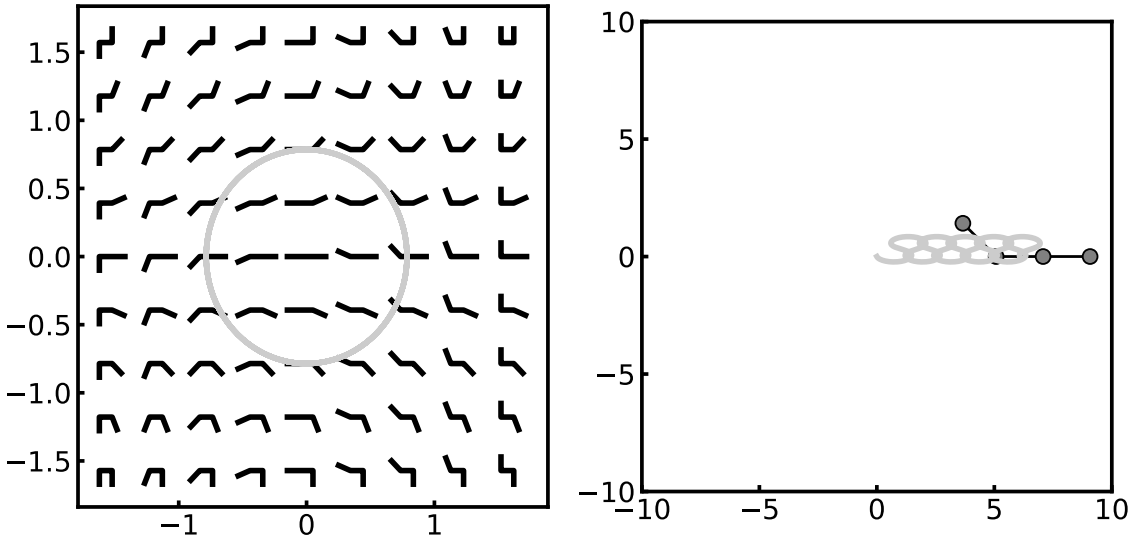
And there is yet one more complication – which is that the forces in the body frame of x- and y- must first be derived in terms of the forces acting on each link. Indeed resistive force theory makes the critical assumptions that one can add the drag and propulsion forces on each link to obtain the overall force. Further, the code includes the anisotropy with respect to the drag experienced when moving a slender link in the direction of flow vs. the drag experience when moving a slender link against the direction of flow. The code itself is complicated so you are encouraged to review the following two sections and perhaps even improve them! The code below is a minimal visualization script. If you want

to see both the configuration space and the world space, check out dropbox. When the code is working you should see something like the following in an animated form,





When complete, the result should look like: such that the complete gait (on the right) includes transient back and forth motion that nonetheless proceeds to the right, increasing in x.



The full simulation model is available in the next boxed section.

Complete simulation code

```
def model_4bead(y,t,pars):
    '''RFT dynamics for a 3 link swimmer
    y -> bx, by, vbx, vby, vtheta, alpha1, alpha2
    A 8-coordinate dynamics, only 6 of which are considered here
    the other two are prescribed by the internal gait dynamics
    '''
    # Coords
    xb = y[0]
    yb = y[1]
    theta = y[2]
    vxb = y[3]
    vyb = y[4]
    vtheta=y[5]
    gait_dyn = pars['gait_function'](t,pars['T'])
    alpha1 = gait_dyn[0]
    alpha2 = gait_dyn[1]
    valpha1 = gait_dyn[2]
    valpha2 = gait_dyn[3]
    L = pars['L']

    # Point positions
    # *----*----*----*
    # 0    1    2    3


    Amat = rotmat(theta+np.pi-alpha1)
    dpos1 = np.matmul(Amat,np.array([2*L,0]))

    Amat = rotmat(theta+alpha2)
    dpos2 = np.matmul(Amat,np.array([2*L,0]))
    bpos = np.array([[xb-L*np.cos(theta)+dpos1[0], yb-L*np.sin(theta)+dpos1[1]],
                     [xb-L*np.cos(theta), yb-L*np.sin(theta)],
                     [xb+L*np.cos(theta), yb+L*np.sin(theta)],
                     [xb+L*np.cos(theta)+dpos2[0], yb+L*np.sin(theta)+dpos2[1]]])
    # Link vectors for anisotropy
    link = np.zeros((4,2))
    link[0,:] = bpos[0,:]-bpos[1,:]
    link[1,:] = bpos[1,:]-bpos[2,:]
    link[2,:] = bpos[2,:]-bpos[1,:]
    link[3,:] = bpos[3,:]-bpos[2,:]

    # Forces on each bead, due to drag
    bvel_vec = np.array([vxb, vyb])
    fparr = np.zeros(np.shape(link))
    fperp = np.zeros(np.shape(link))
    for i in range(4):
        vec_projection = np.dot(bvel_vec, link[i,:])/(2*L)*link[i,:]/(2*L)
        vec_rejection = bvel_vec- np.dot(bvel_vec, link[i,:])/(2*L)**2*link[i,:]
        fparr[i,:] = -pars['kparr']*vec_projection
        fperp[i,:] = -pars['kperp']*vec_rejection

    # Forces on the movable beads
    fparr_bead = np.zeros((2,2))
    fperp_bead = np.zeros((2,2))

    bead_vangle1 = vec_to_angle(link[0,0],link[0,1])-np.pi/2*np.sign(valpha1)
    bead_vvec1 = angle_to_vec(bead_vangle1)
    vec_projection = np.dot(bead_vvec1, link[0,:])/(2*L)*link[0,:]/(2*L)
    vec_rejection = bead_vvec1-np.dot(bead_vvec1, link[0,:])/(2*L)**2*link[0,:]
    fparr_bead[0,:] = -pars['kparr']*np.abs(valpha1)*2*L*vec_projection
    fperp_bead[0,:] = -pars['kperp']*np.abs(valpha1)*2*L*vec_rejection

    bead_vangle2 = vec_to_angle(link[3,0],link[3,1])+np.pi/2*np.sign(valpha2)
    bead_vvec2 = angle_to_vec(bead_vangle2)
    vec_projection = np.dot(bead_vvec2, link[3,:])/(2*L)*link[3,:]/(2*L)
    vec_rejection = bead_vvec2-np.dot(bead_vvec2, link[3,:])/(2*L)**2*link[3,:]
    fparr_bead[1,:]=-pars['kparr']*np.abs(valpha2)*2*L*vec_projection
    fperp_bead[1,:]=-pars['kperp']*np.abs(valpha2)*2*L*vec_rejection


    # Total force
    F = np.sum(fparr,axis=0)+np.sum(fperp,axis=0)+np.sum(fparr_bead,axis=0)+\
    np.sum(fperp_bead,axis=0)
```

```
#Equations of motion
dydt = np.zeros(6)
dydt[0]=vxb
dydt[1]=vyb
dydt[2]=vtheta
dydt[3]=pars['damping']*F[0]/pars['M']
dydt[4]=pars['damping']*F[1]/pars['M']
dydt[5]=0

return dydt
```

Master script

```
pars={}
pars['L']=1
pars['kparr']=0.2
pars['kperp']=2
pars['max_angle']=np.pi/4
pars['damping']=1
pars['M']=0.1
pars['gait_function']= asym_gait
pars['T']=40
pars['cnt']=0

# Simulate
y0=np.zeros(6) #6 equations, all in world frame
t = np.arange(0,200.2,0.2)
y = integrate.odeint(model_4bead,y0,t,args=(pars,))

# Recover the gaits
alpha1 = np.zeros(np.shape(t))
alpha2 = np.zeros(np.shape(t))
valpha1 = np.zeros(np.shape(t))
valpha2 = np.zeros(np.shape(t))
for i in range(len(t)):
    gait_dyn = pars['gait_function'](t[i],pars['T'])
    alpha1[i]=gait_dyn[0]
    alpha2[i]=gait_dyn[1]
    valpha1[i]=gait_dyn[2]
    valpha2[i]=gait_dyn[3]

#Visualize swimming
#Do not forget to run the command %matplotlib qt first
fig, axes = plt.subplots(1,2,figsize=(10,5))

#also show the shape space
ax=axes[0]
L = 0.05 # Artificially create 'small' swimmers for visualization
dax = np.pi/8
day = np.pi/8
alpha1_vec = np.arange(-np.pi/2,np.pi/2+dax,dax)
alpha2_vec = np.arange(-np.pi/2,np.pi/2+day,day)
alpha1_grid, alpha2_grid = np.meshgrid(alpha1_vec, alpha2_vec)
for i in range(len(alpha1_vec)):
    for j in range(len(alpha2_vec)):
        pos = body_coords([alpha1_grid[j,i],alpha2_grid[j,i]],L) #Configuration
        vis_pos = pos + np.array([alpha1_grid[j,i],alpha2_grid[j,i]])
        # Shift placement to alpha-1/alpha-2
        ax.plot(vis_pos[:,0],vis_pos[:,1],'k',linewidth=3)
ax.plot(alpha1,alpha2,c='r',linewidth=3)
plt.setp(ax.spines.values(),linewidth=2)
ax.tick_params(labelsize=15,direction='in',width=2)

ax = axes[1]
ax.set_xlim(-10,10)
ax.set_ylim(-10,10)
ax.set_aspect('equal')
plt.setp(ax.spines.values(),linewidth=2)
ax.tick_params(labelsize=15,direction='in',width=2)


#Go back to the swimming
for i in range(len(t)):
    ax.clear()
    ax.set_xlim(-10,10)
    ax.set_ylim(-10,10)
    ax.set_aspect('equal')
    pos = body_coords([alpha1[i],alpha2[i]],pars['L'])
    rpos = body_rotate(pos,y[i,2])
    vis_pos = np.array([y[i,0], y[i,1]])*np.ones(np.shape(rpos))+rpos
    ax.plot(vis_pos[:,0], vis_pos[:,1],'k',
            linestyle='-',
            marker='o',
            markeredgecolor='k',
            markerfacecolor=[0.5,0.5,0.5],
            markersize=8)

    plt.pause(0.001)
ax.plot(y[:,0],y[:,1],c='r',linewidth=3)
plt.show()
```

V. THE GREATEST GAIT OF ALL

Now that you have a working code, you should aim for the following:

- Try out the symmetric and anti-symmetric gaits. Which of them leads to translation and why?
- By changing the magnitude of angular change, can you modify the horizontal translation per gait cycle? Is there an optimal angle for the anti-symmetric gait?
- How does the mass change the ability of the 3-link swimmer to self-propel?
- How much does anisotropy matter? That is, if one sets the parallel and perpendicular drag forces equal, does that decrease or increase the amount of translation per gait cycle?

Finally, if you have time, see if you can outdo the anti-symmetric gait and measure the translation per gait cycle. If you were the 3-link swimmer, how would you want to move? These targets form the basis for the homework and are central to the development of theory, simulations, analysis of living systems, and the construction of real-world robots that leverage 'geometric mechanics'.